

Matplotlib is a low level graph plotting library in python that serves as a visualization utility. Matplotlib was created by John D. Hunter, Matplotlib has become a fundamental tool for data visualization in Python, extensively used by data scientists, researchers and engineers worldwide. Matplotlib is open source and we can use it freely. Matplotlib is mostly written in python, a few segments are written in C, Objective-C and Javascript for Platform compatibility.

What is Matplotlib in Python used for

With Matplotlib, we can perform a wide range of visualization tasks, including:

- Creating basic plots such as line, bar and scatter plots.
- Customizing plots with labels, titles, legends and color schemes.
- Adjusting figure size, layout and aspect ratios.
- Saving plots in various formats like PNG, PDF and SVG.
- Combining multiple plots into subplots for better data representation.
- Creating interactive plots using the widget module.

Installation of Matplotlib

If you have **Python** and **PIP** already installed on a system, then installation of Matplotlib is very easy. Install it using this command:

```
C:\Users\Your Name>pip install matplotlib
```

Import Matplotlib

Once Matplotlib is installed, import it in your applications by adding the import module statement:

```
import matplotlib
```

Checking Matplotlib Version

The version string is stored under `__version__` attribute.

```
import matplotlib  
print(matplotlib.__version__)
```

Pyplot

Most of the Matplotlib utilities lies under the **pyplot** submodule, and are usually imported under the **plt** alias:

```
import matplotlib.pyplot as plt
```

Now the Pyplot package can be referred to as `plt`.

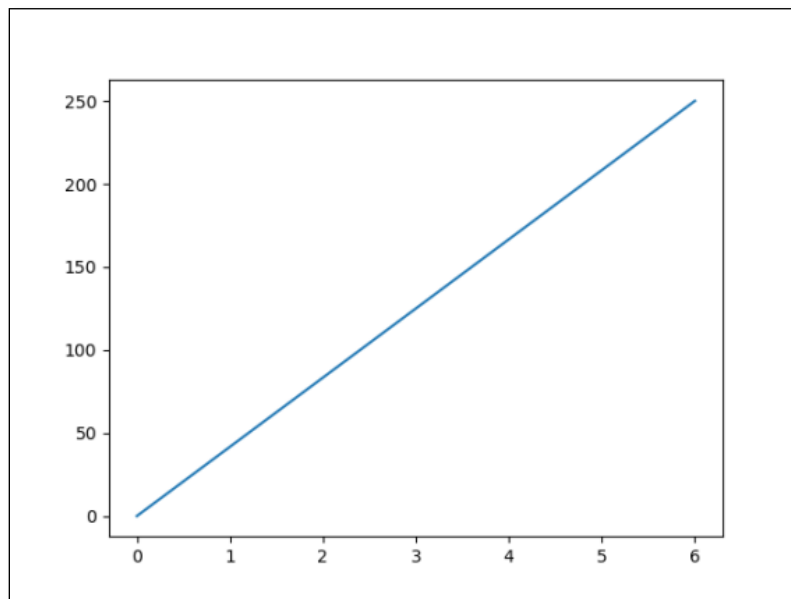
Example of a Plot in Matplotlib:

Draw a line in a diagram from position(0,0) to position (6,250):

```
import matplotlib.pyplot as plt  
import numpy as np
```

```
xpoints = np.array([0, 6])  
ypoints = np.array([0, 250])
```

```
plt.plot(xpoints, ypoints)  
plt.show()
```



Steps to Use Pyplot

- **Import Matplotlib:** Start by importing matplotlib.pyplot as plt.
- **Create Data:** Prepare your data in the form of lists or arrays.
- **Plot Data:** Use plt.plot() to create the plot.
- **Customize Plot:** Add titles, labels, and other elements using methods like plt.title(), plt.xlabel(), and plt.ylabel().
- **Display Plot:** Use plt.show() to display the plot.

Multiple Points

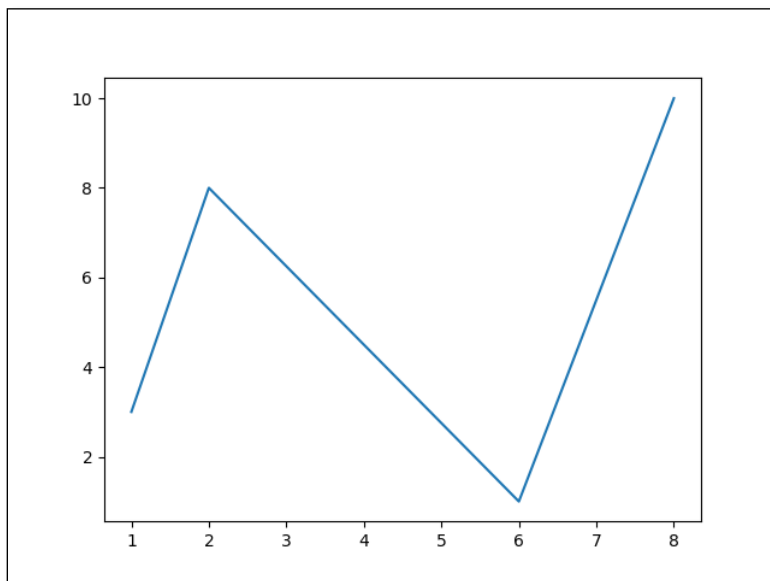
You can plot as many points as you like, just make sure you have the same number of points in both axis.

Draw a line in a diagram from position (1,3) to (2,8) then to (6,1) and finally to position (8,10)

```
import matplotlib.pyplot as plt
import numpy as np
```

```
xpoints = np.array([1, 2, 6, 8])
ypoints = np.array([3, 8, 1, 10])
```

```
plt.plot(xpoints, ypoints)
plt.show()
```



Default X-Points

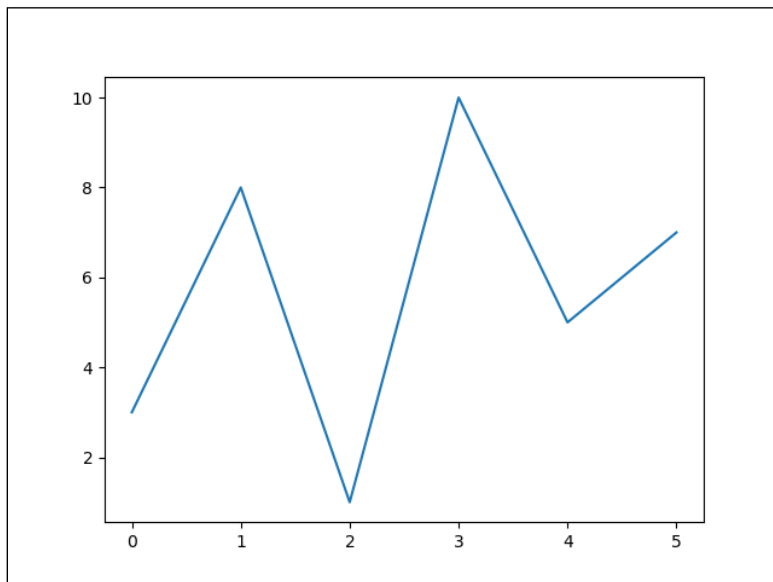
If we do not specify the points on the x-axis, they will get the default values 0, 1, 2, 3 etc., depending on the length of the y-points.

Plotting without x-points:

```
import matplotlib.pyplot as plt
import numpy as np

ypoints = np.array([3, 8, 1, 10, 5, 7])

plt.plot(ypoints)
plt.show()
```



Markers

You can use the keyword argument **marker** to emphasize each point with a specified marker:

Mark each point with a circle:

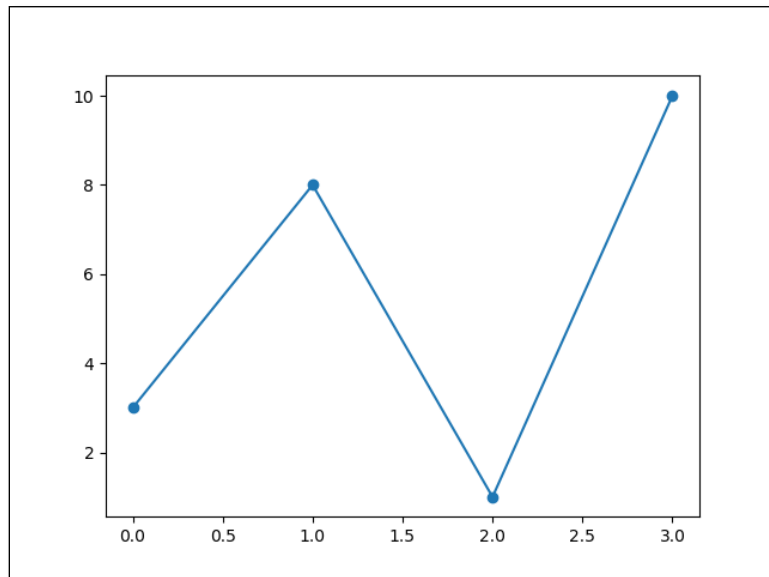
```
import matplotlib.pyplot as plt
```

```
import numpy as np
```

```
ypoints = np.array([3, 8, 1, 10])
```

```
plt.plot(ypoints, marker = 'o')
```

```
plt.show()
```



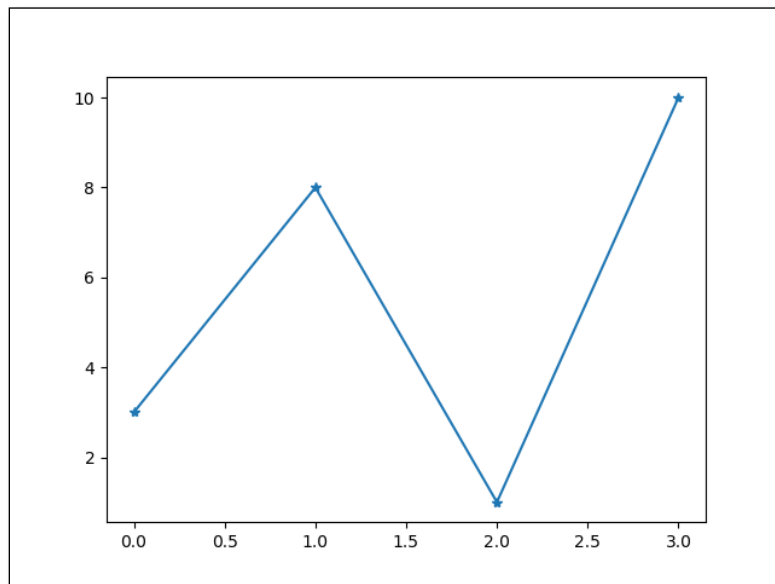
Mark each point with a star:

```
import matplotlib.pyplot as plt
```

```
import numpy as np
```

```
ypoints = np.array([3, 8, 1, 10])
```

```
plt.plot(ypoints, marker = '*')  
plt.show()
```



Marker Reference

You can choose any of these markers:

Marker	Description
'o'	Circle
'*'	Star
'.'	Point
','	Pixel
'x'	X
'X'	X (filled)
'+'	Plus
'P'	Plus (filled)
's'	Square

'D'	Diamond
'd'	Diamond (thin)
'p'	Pentagon
'H'	Hexagon
'h'	Hexagon
'v'	Triangle Down
'^'	Triangle Up
'<'	Triangle Left
'>'	Triangle Right
'1'	Tri Down
'2'	Tri Up
'3'	Tri Left
'4'	Tri Right
' '	Vline
'_'	Hline

Linestyle

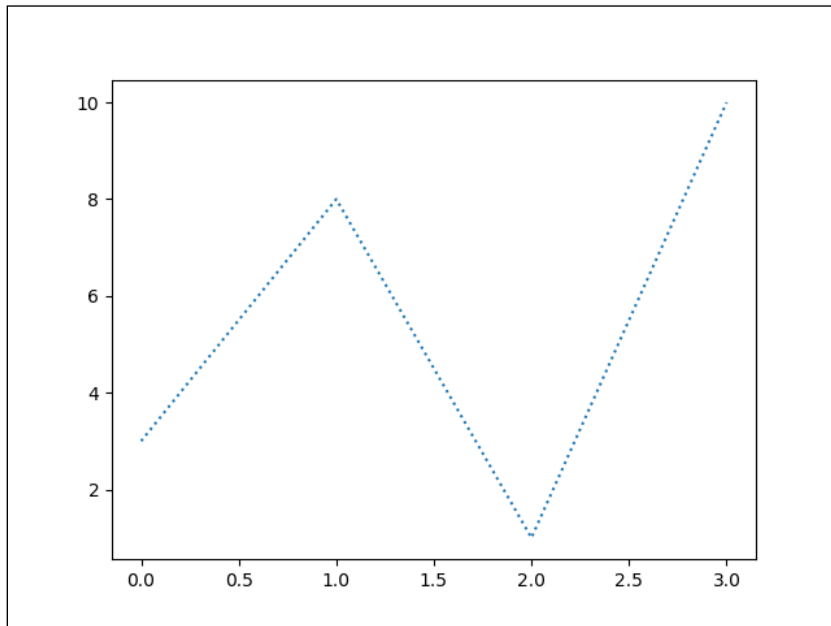
You can use the keyword argument **linestyle**, or shorter **ls**, to change the style of the plotted line:

Use a dotted line:

```
import matplotlib.pyplot as plt
import numpy as np

ypoints = np.array([3, 8, 1, 10])
```

```
plt.plot(ypoints, linestyle = 'dotted')  
plt.show()
```



Shorter Syntax

The line style can be written in a shorter syntax:

linestyle can be written as **ls**.

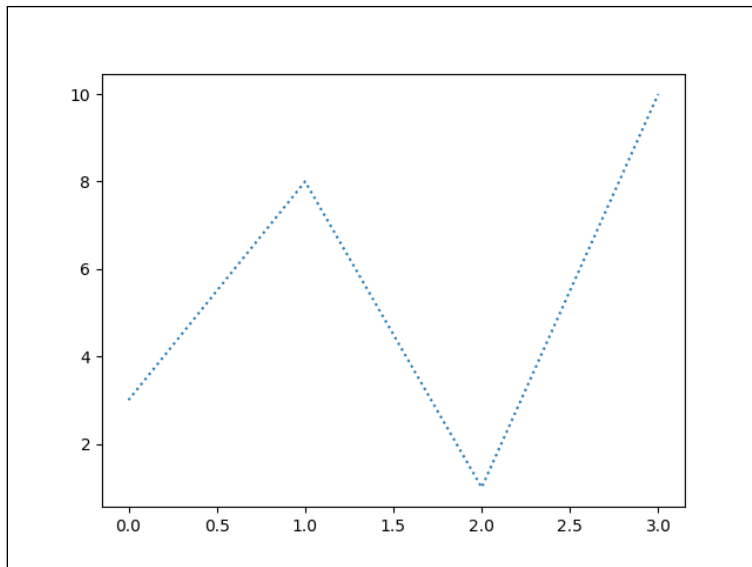
dotted can be written as **:**

dashed can be written as **--**

```
import matplotlib.pyplot as plt  
import numpy as np
```

```
ypoints = np.array([3, 8, 1, 10])
```

```
plt.plot(ypoints, ls = ':')  
plt.show()
```



Line Reference

Line Syntax	Description
'-'	Solid line
'.'	Dotted line
'--'	Dashed line
'-.'	Dashed/dotted line

Line Color

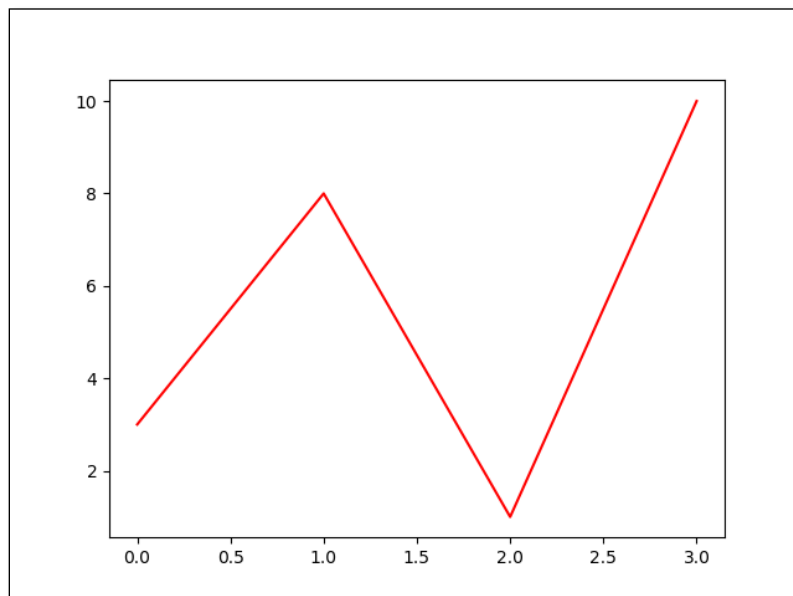
You can use the keyword argument **color** or the shorter **c** to set the color of the line:

Set the line color to red:

```
import matplotlib.pyplot as plt
import numpy as np

ypoints = np.array([3, 8, 1, 10])

plt.plot(ypoints, color = 'r')
plt.show()
```



Color Reference

Color Syntax	Description
--------------	-------------

'r'	Red
'g'	Green
'b'	Blue
'c'	Cyan
'm'	Magenta
'y'	Yellow
'k'	Black
'w'	White

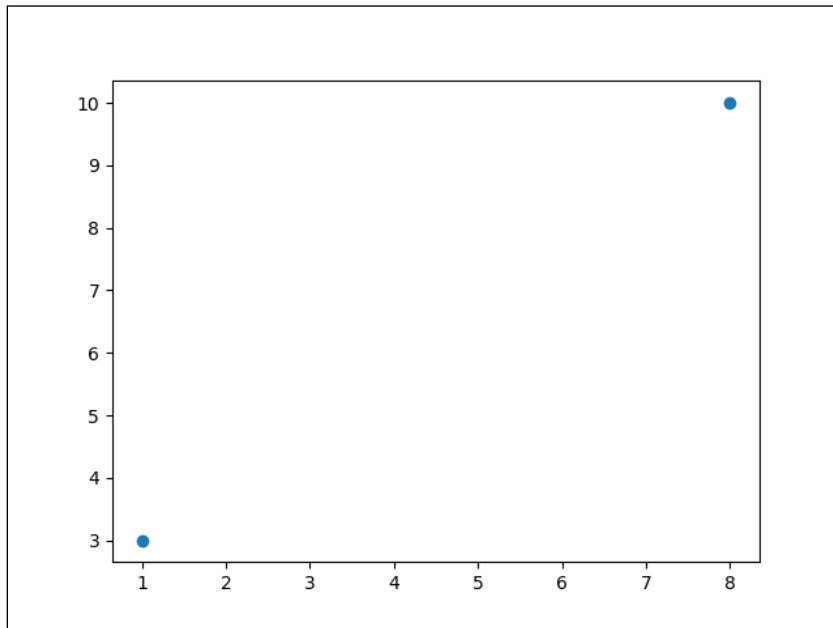
Plotting Without Line

Draw two points in the diagram, one at position (1, 3) and one in position (8, 10):

```
import matplotlib.pyplot as plt
import numpy as np
```

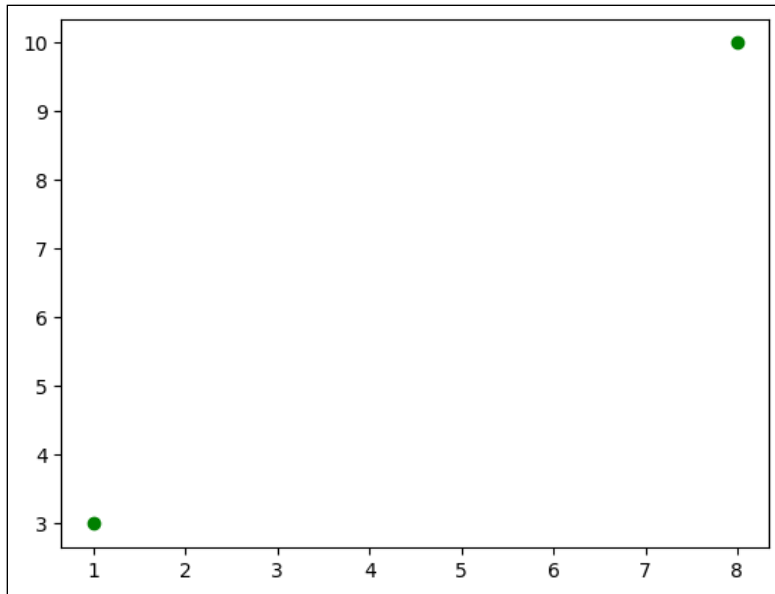
```
xpoints = np.array([1, 8])
ypoints = np.array([3, 10])
```

```
plt.plot(xpoints, ypoints, 'o')  
plt.show()
```



Marker colour green

```
import matplotlib.pyplot as plt  
import numpy as np  
  
xpoints = np.array([1, 8])  
ypoints = np.array([3, 10])  
  
plt.plot(xpoints, ypoints, 'og')  
plt.show()
```



Matplotlib Labels and Title

Create Labels for a Plot

With Pyplot, you can use the **xlabel()** and **ylabel()** functions to set a label for the x- and y-axis.

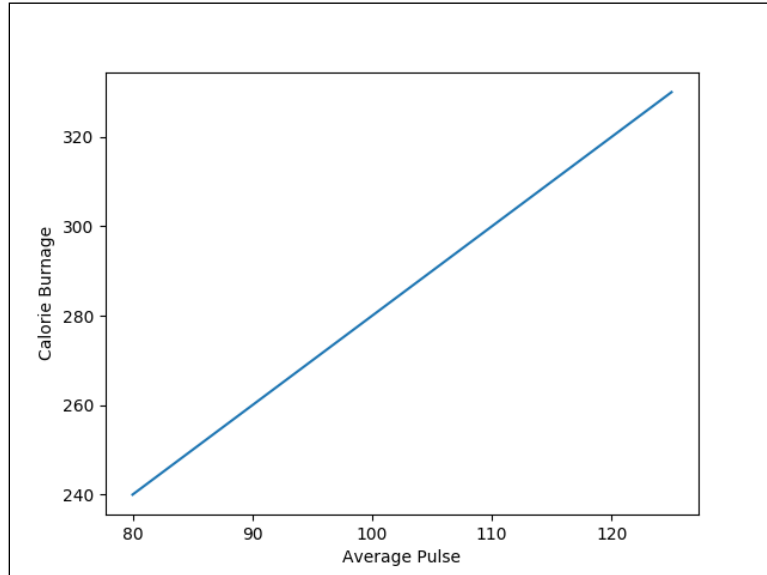
```
import numpy as np
import matplotlib.pyplot as plt

x = np.array([80, 85, 90, 95, 100, 105, 110, 115, 120, 125])
y=np.array([240, 250, 260, 270, 280, 290, 300, 310, 320,
330])

plt.plot(x, y)
```

```
plt.xlabel("Average Pulse")
plt.ylabel("Calorie Burnage")

plt.show()
```



Create a Title for a Plot

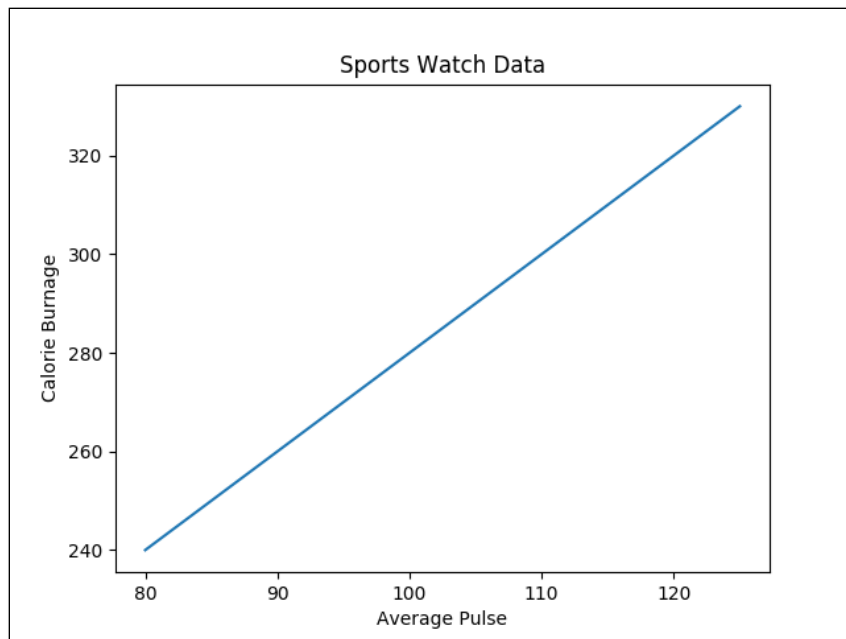
With Pyplot, you can use the **title()** function to set a title for the plot.

```
import numpy as np
import matplotlib.pyplot as plt

x = np.array([80, 85, 90, 95, 100, 105, 110, 115, 120, 125])
y=np.array([240, 250, 260, 270, 280, 290, 300, 310, 320,
330])
```

```
plt.plot(x, y)
plt.title("Sports Watch Data")
plt.xlabel("Average Pulse")
plt.ylabel("Calorie Burnage")

plt.show()
```



Set Font Properties for Title and Labels

You can use the **fontdict** parameter in `xlabel()`, `ylabel()`, and `title()` to set font properties for the title and labels.

```
import numpy as np
import matplotlib.pyplot as plt
```

```
x = np.array([80, 85, 90, 95, 100, 105, 110, 115, 120, 125])
```

```
y=np.array([240, 250, 260, 270, 280, 290, 300, 310, 320,  
330])
```

```
font1 = {'family':'serif','color':'blue','size':20}
```

```
font2 = {'family':'serif','color':'darkred','size':15}
```

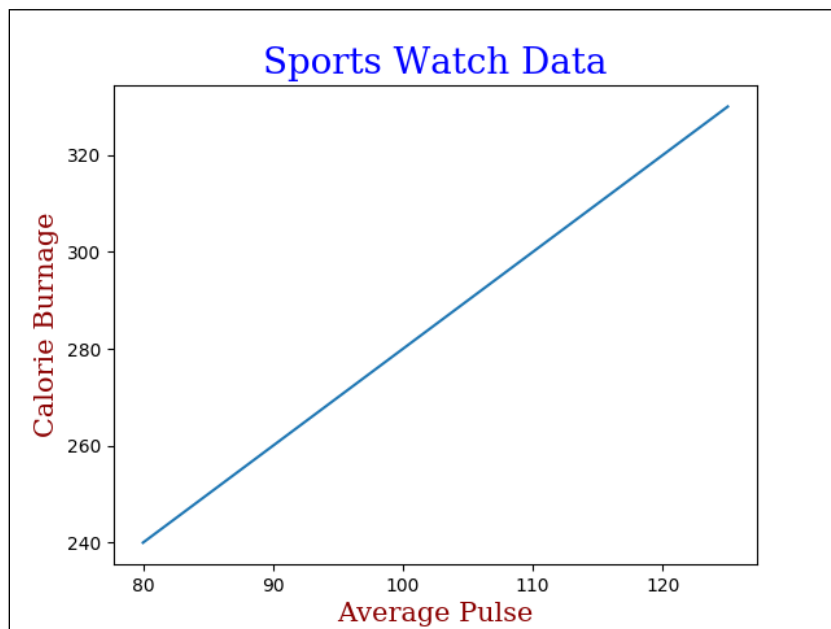
```
plt.title("Sports Watch Data", fontdict = font1)
```

```
plt.xlabel("Average Pulse", fontdict = font2)
```

```
plt.ylabel("Calorie Burnage", fontdict = font2)
```

```
plt.plot(x, y)
```

```
plt.show()
```



Position the Title

You can use the **loc** parameter in `title()` to position the title.

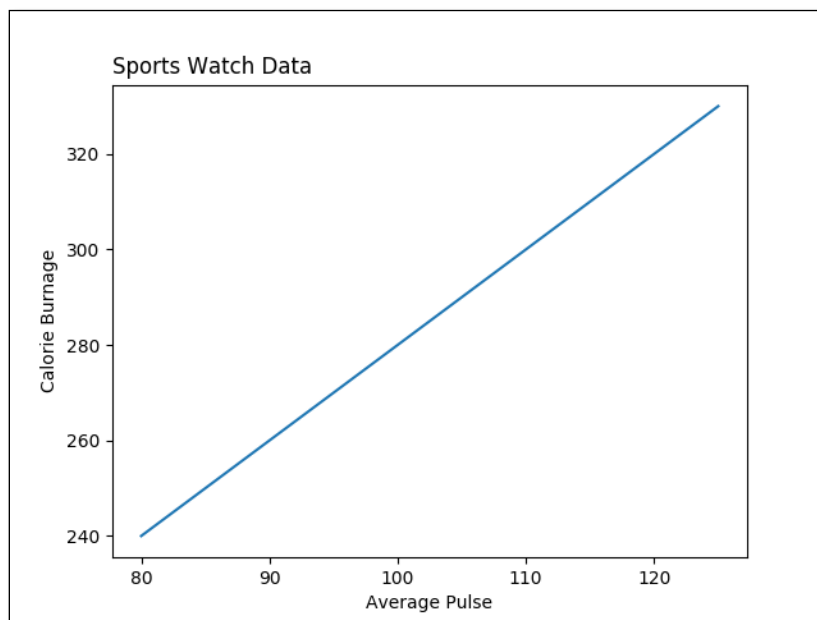
Legal values are: 'left', 'right', and 'center'. Default value is 'center'.

```
import numpy as np
import matplotlib.pyplot as plt

x = np.array([80, 85, 90, 95, 100, 105, 110, 115, 120, 125])
y=np.array([240, 250, 260, 270, 280, 290, 300, 310, 320,
330])

plt.title("Sports Watch Data", loc = 'left')
plt.xlabel("Average Pulse")
plt.ylabel("Calorie Burnage")

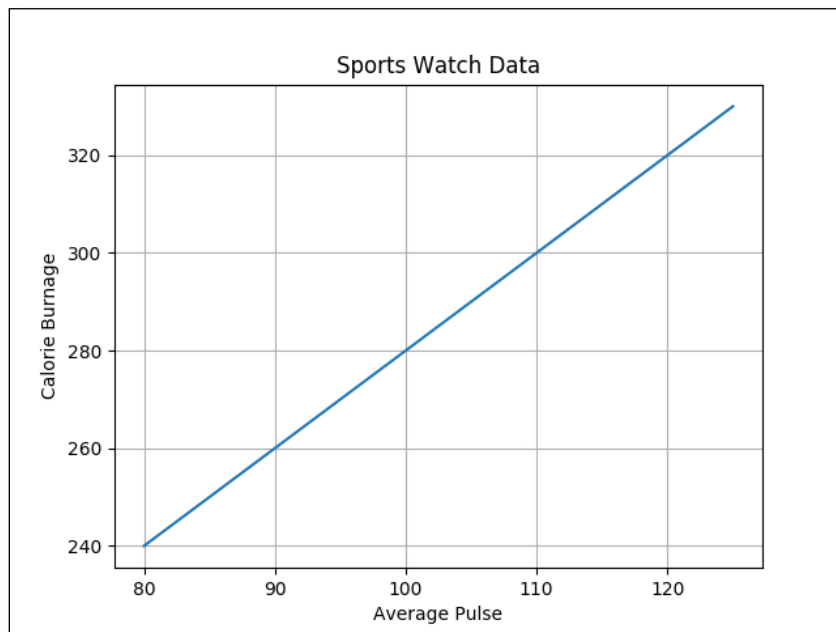
plt.plot(x, y)
plt.show()
```



Add Grid Lines to a Plot

With Pyplot, you can use the **grid()** function to add grid lines to the plot.

```
import numpy as np
import matplotlib.pyplot as plt
x = np.array([80, 85, 90, 95, 100, 105, 110, 115, 120, 125])
y=np.array([240, 250, 260, 270, 280, 290, 300, 310, 320, 330])
plt.title("Sports Watch Data")
plt.xlabel("Average Pulse")
plt.ylabel("Calorie Burnage")
plt.plot(x, y)
plt.grid()
plt.show()
```



Specify Which Grid Lines to Display

You can use the `axis` parameter in the **grid()** function to specify which grid lines to display.

Legal values are: 'x', 'y', and 'both'. Default value is 'both'.

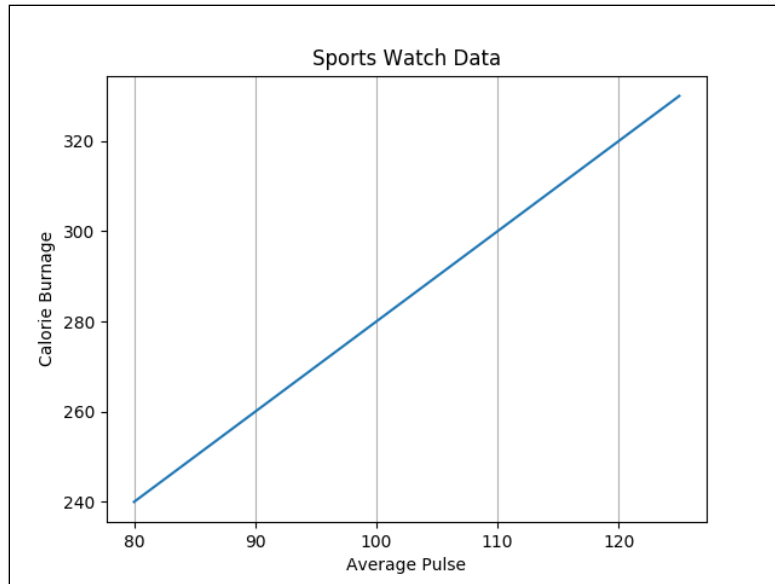
Display only grid lines for the x-axis:

```
import numpy as np
import matplotlib.pyplot as plt

x = np.array([80, 85, 90, 95, 100, 105, 110, 115, 120, 125])
y = np.array([240, 250, 260, 270, 280, 290, 300, 310, 320, 330])

plt.title("Sports Watch Data")
plt.xlabel("Average Pulse")
plt.ylabel("Calorie Burnage")

plt.plot(x, y)
plt.grid(axis = 'x')
plt.show()
```



Set Line Properties for the Grid

You can also set the line properties of the grid, like this: `grid(color = 'color', linestyle = 'linestyle', linewidth = number)`.

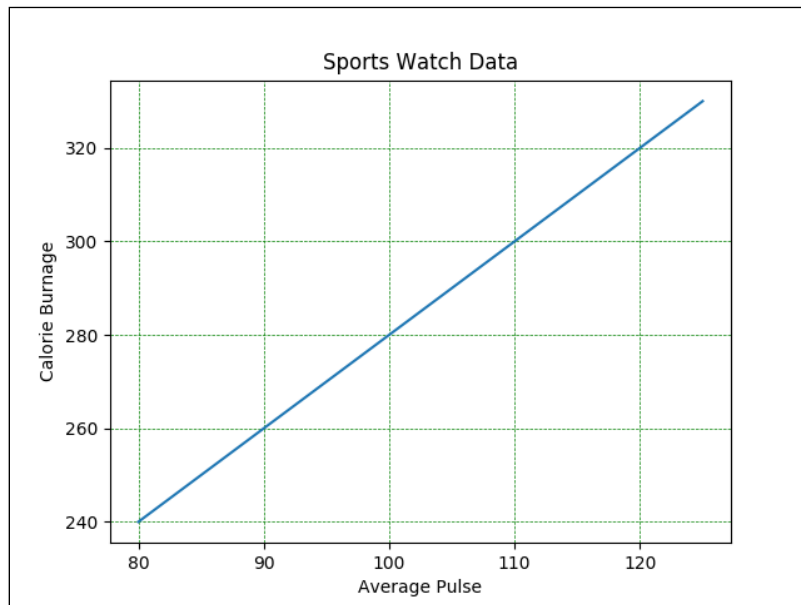
```
import numpy as np
import matplotlib.pyplot as plt

x = np.array([80, 85, 90, 95, 100, 105, 110, 115, 120, 125])
y=np.array([240, 250, 260, 270, 280, 290, 300, 310, 320,
330])

plt.title("Sports Watch Data")
plt.xlabel("Average Pulse")
plt.ylabel("Calorie Burnage")

plt.plot(x, y)
```

```
plt.grid(color = 'green', linestyle = '--', linewidth = 0.5)
plt.show()
```



Display Multiple Plots

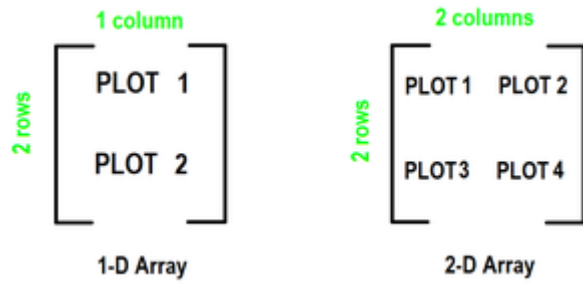
With the **subplot()** function you can draw multiple plots in one figure:

subplot() Function

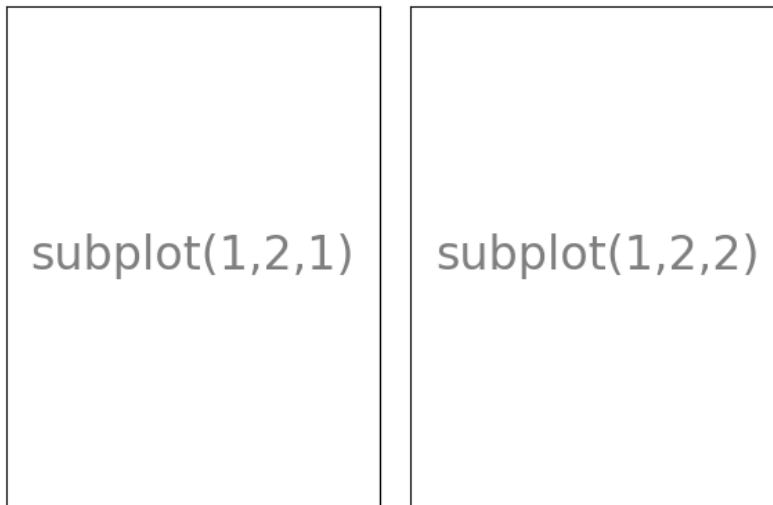
The subplot() function takes three arguments that describes the layout of the figure.

The layout is organized in rows and columns, which are represented by the *first* and *second* argument.

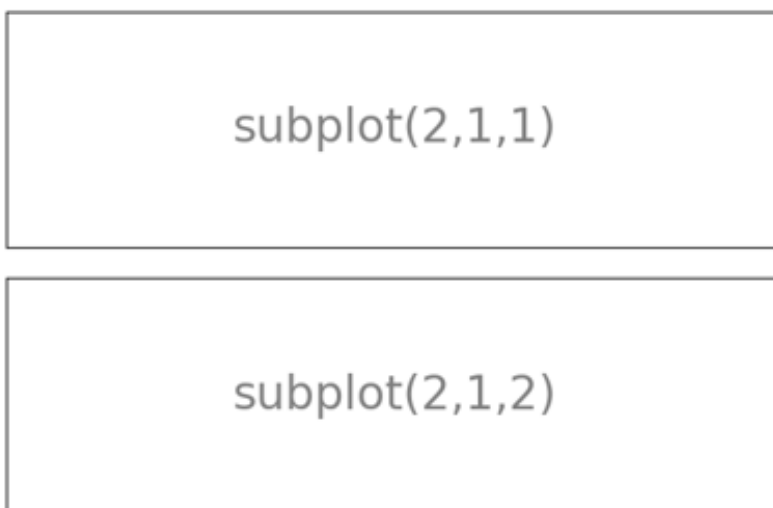
The third argument represents the index of the current plot.



Plot two graph with 1 row and 2 columns



Plot two graph with 2 row and 1 columns



Plot four graph with 2 row and 2 columns



Draw two plots

```
import matplotlib.pyplot as plt
import numpy as np
```

#plot 1:

```
x = np.array([0, 1, 2, 3])
y = np.array([3, 8, 1, 10])
```

```
plt.subplot(1, 2, 1)
plt.plot(x,y)
```

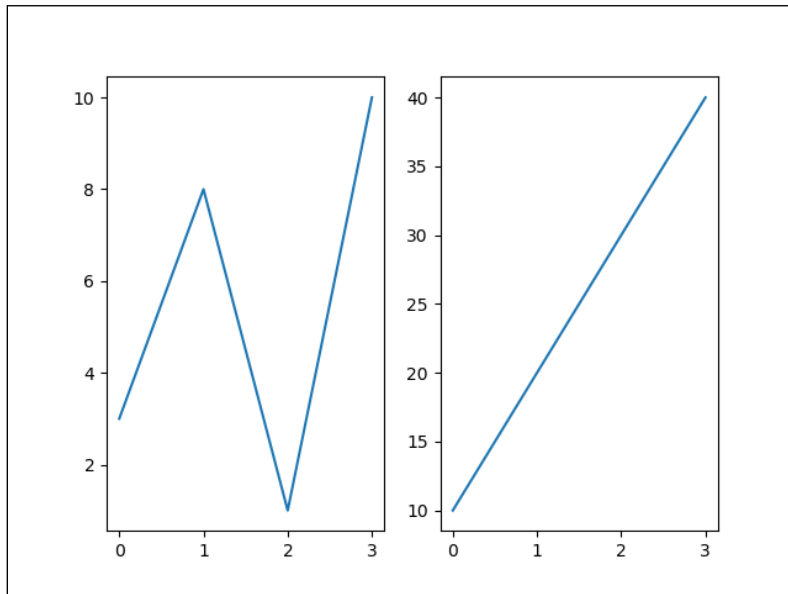
#plot 2:

```
x = np.array([0, 1, 2, 3])
y = np.array([10, 20, 30, 40])
```

```
plt.subplot(1, 2, 2)
```

```
plt.plot(x,y)
```

```
plt.show()
```



Draw two plots on top of each other

```
import matplotlib.pyplot as plt
```

```
import numpy as np
```

```
#plot 1:
```

```
x = np.array([0, 1, 2, 3])
```

```
y = np.array([3, 8, 1, 10])
```

```
plt.subplot(2, 1, 1)
```

```
plt.plot(x,y)
```

```
#plot 2:
```

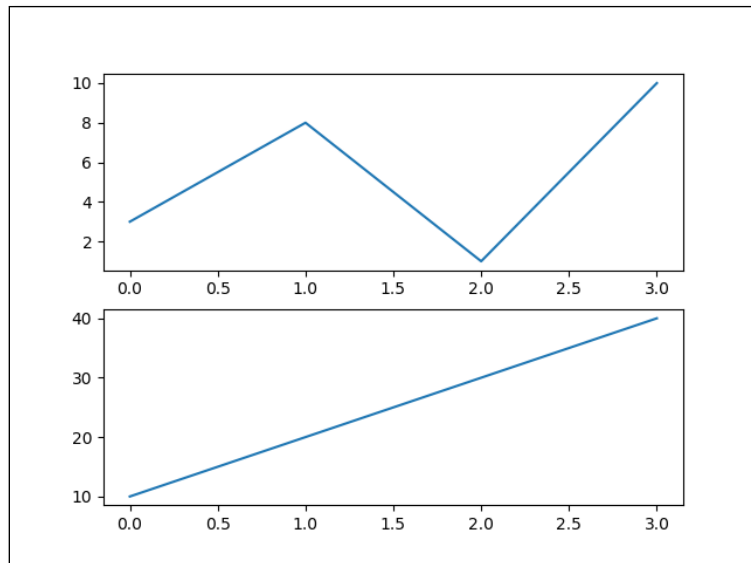
```
x = np.array([0, 1, 2, 3])
```

```
y = np.array([10, 20, 30, 40])
```

```
plt.subplot(2, 1, 2)
```

```
plt.plot(x,y)
```

```
plt.show()
```



Draw 6 plots:

```
import matplotlib.pyplot as plt
```

```
import numpy as np
```

```
x = np.array([0, 1, 2, 3])  
y = np.array([3, 8, 1, 10])
```

```
plt.subplot(2, 3, 1)  
plt.plot(x,y)
```

```
x = np.array([0, 1, 2, 3])  
y = np.array([10, 20, 30, 40])
```

```
plt.subplot(2, 3, 2)  
plt.plot(x,y)
```

```
x = np.array([0, 1, 2, 3])  
y = np.array([3, 8, 1, 10])
```

```
plt.subplot(2, 3, 3)  
plt.plot(x,y)
```

```
x = np.array([0, 1, 2, 3])  
y = np.array([10, 20, 30, 40])
```

```
plt.subplot(2, 3, 4)  
plt.plot(x,y)
```

```
x = np.array([0, 1, 2, 3])  
y = np.array([3, 8, 1, 10])
```

```
plt.subplot(2, 3, 5)
```

```
plt.plot(x,y)
```

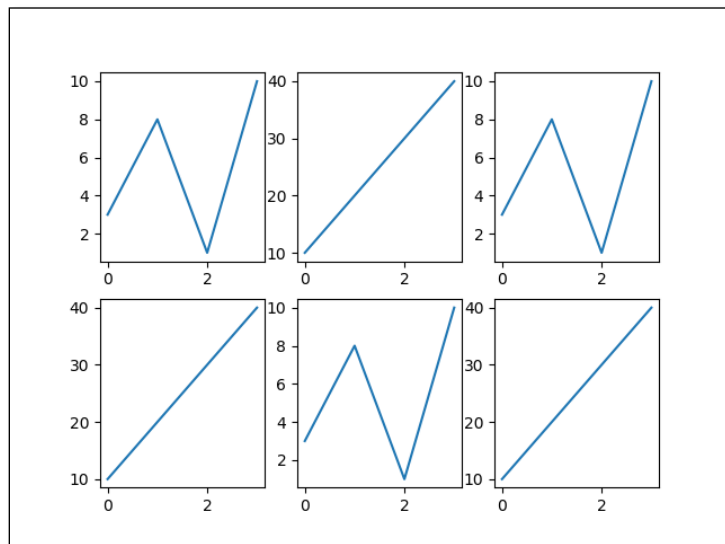
```
x = np.array([0, 1, 2, 3])
```

```
y = np.array([10, 20, 30, 40])
```

```
plt.subplot(2, 3, 6)
```

```
plt.plot(x,y)
```

```
plt.show()
```



Title

You can add a title to each plot with the **title()** function:

```
import matplotlib.pyplot as plt
import numpy as np
```

```
#plot 1:
```

```
x = np.array([0, 1, 2, 3])
y = np.array([3, 8, 1, 10])
```

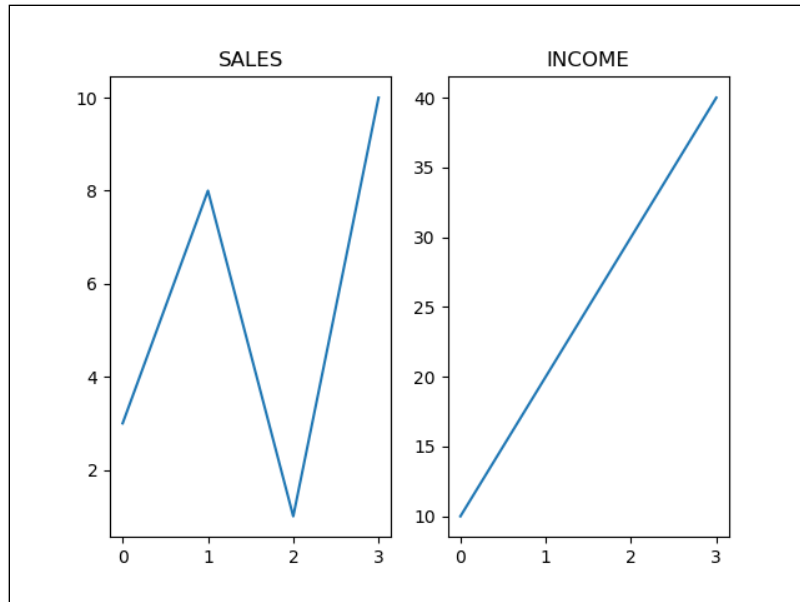
```
plt.subplot(1, 2, 1)
plt.plot(x,y)
plt.title("SALES")
```

```
#plot 2:
```

```
x = np.array([0, 1, 2, 3])
y = np.array([10, 20, 30, 40])
```

```
plt.subplot(1, 2, 2)
plt.plot(x,y)
plt.title("INCOME")
```

```
plt.show()
```



Super Title

You can add a title to the entire figure with the **suptitle()** function:

```
import matplotlib.pyplot as plt
import numpy as np
```

```
x1 = np.array([0, 1, 2, 3])
y1 = np.array([3, 8, 1, 10])
```

```
x2 = np.array([0, 1, 2, 3])
y2 = np.array([10, 20, 30, 40])
```

```
plt.figure(figsize=(10, 6))
```

```
plt.subplot(2, 3, 1)
```

```
plt.plot(x1, y1)
plt.title("Plot 1")
```

```
plt.subplot(2, 3, 2)
plt.plot(x2, y2)
plt.title("Plot 2")
```

```
plt.subplot(2, 3, 3)
plt.plot(x1, y1)
plt.title("Plot 3")
```

```
plt.subplot(2, 3, 4)
plt.plot(x2, y2)
plt.title("Plot 4")
```

```
plt.subplot(2, 3, 5)
plt.plot(x1, y1)
plt.title("Plot 5")
```

```
plt.subplot(2, 3, 6)
plt.plot(x2, y2)
plt.title("Plot 6")
plt.suptitle("Multiple Subplots Example", fontsize=14,
fontweight='bold')
```

```
plt.tight_layout(rect=[0, 0, 1, 0.95]) # Adjust layout to  
fit the super title  
plt.show()
```

